

CNN Lab Module for Interns

Module Title: Practical Introduction to Convolutional Neural Networks (CNNs)

Prerequisite Knowledge:

- Basics of Python and NumPy
- Basic understanding of how neural networks work (ANNs)
- Understanding of image data (grayscale vs. RGB)

Recommended Introductory Resources:

1. YouTube: ["What is a Convolutional Neural Network \(CNN\)?"](#) by 3Blue1Brown – *Highly intuitive visual explanation*
 2. YouTube: ["Convolutional Neural Networks \(CNN\) - Deep Learning" by Simplilearn](#)
 3. Blog: [A Beginner's Guide to CNNs – Towards Data Science](#)
-

Lab 01: Image Representation and Manual Convolution

Objective:

To understand how images are represented numerically, and how convolution filters work manually before using any library.

Tasks:

1. Load and visualize a grayscale image using PIL and matplotlib.
 - What is the shape and data type of the loaded image?
 - How are pixel values represented?
2. Load and visualize a colored (RGB) image.
 - Separate the R, G, B channels.
 - Visualize each channel separately. What differences do you observe?

3. Convert RGB image to grayscale manually using weighted average ($0.2989 * R + 0.5870 * G + 0.1140 * B$).
 - Compare this with built-in grayscale conversion.
4. Implement a 3x3 edge detection kernel (e.g., Sobel or Laplacian) from scratch using NumPy.
 - Apply it to a small grayscale image.
 - Visualize input vs. convolved output.
 - What does the kernel do?
5. Implement zero padding manually and explain its effect.
 - Try padding = 0 and padding = 1. What changes?
6. Apply stride = 2 and observe how output size changes.
 - Compute output shape mathematically and verify with code.
7. Implement and compare following kernels:
 - Identity kernel
 - Sharpening filter
 - Gaussian blur
 - Edge detector (horizontal and vertical)
8. Write a function to perform convolution for any 2D input and 3x3 kernel.
 - Add options for padding and stride.
 - Test it on various filters and images.
9. Analyze how convolving the same image with different kernels gives different information (edges, textures, etc.).
10. Bonus: Implement Max Pooling manually on 2x2 windows.

- Compare the original and pooled image.
- What is lost? What is retained?

Deliverables:

- Jupyter Notebook with step-by-step solutions and outputs
 - Observations written after each experiment
 - Comparison of kernels and visual insights
-

Lab 02: Building a CNN from Scratch Using NumPy

Lab Title: Implementing a Basic Convolutional Neural Network Without Frameworks

Problem Statement:

Classify handwritten digits from the MNIST dataset using a CNN built from scratch using only NumPy.

Objective:

To gain hands-on experience in implementing the core components of a Convolutional Neural Network using low-level code without using PyTorch or TensorFlow.

Tasks and Guiding Questions:**Task 1: Load and Prepare the MNIST Dataset**

- Use `keras.datasets.mnist` or download MNIST manually.
- Normalize image pixel values to be between 0 and 1.
- Flatten the label set into one-hot encoded vectors.

Questions:

- What is the shape of a single image in MNIST?

- Why is normalization important before feeding to a neural network?

Task 2: Implement the Convolution Layer

- Use NumPy to implement a basic 2D convolution operation (no padding, stride = 1).
- Create a method `convolve(image, kernel)` that handles one channel.
- Use filters with random weights.

Questions:

- How does kernel size affect output shape?
- How can we generalize this for multi-channel images?

Task 3: Implement ReLU Activation

- Define a `relu(x)` function that sets all negative values to zero.
- Apply ReLU after each convolution.

Questions:

- What is the purpose of ReLU in CNNs?
- What would happen if we skip this activation?

Task 4: Implement Max Pooling Layer

- Define a method `max_pool(image, size=2, stride=2)`.
- Reduce each feature map using non-overlapping windows.

Questions:

- What is the impact of max pooling on the image size?
- Why do we use pooling instead of just stacking more convolutions?

Task 5: Implement the Fully Connected Layer

- Flatten the output from the last pooling layer.
- Manually initialize weight matrix and bias vector.
- Perform dot product with weights + bias.

Questions:

- How do we calculate the number of weights in a fully connected layer?
- Why is flattening needed before feeding to dense layers?

Task 6: Implement Softmax and Loss Function

- Write a softmax function to convert outputs into probabilities.
- Implement cross-entropy loss function.

Questions:

- Why do we use softmax instead of sigmoid for multi-class classification?
- What does the loss value represent during training?

Task 7: Training Loop

- Perform forward pass using all layers.
- Calculate loss.
- (Optional/Advanced) Implement backpropagation for gradients.
- For now, only implement one forward pass per batch.

Questions:

- What values change after each forward pass?

- What would be needed to implement backpropagation?

Task 8: Test and Evaluate

- Run trained model (even if unoptimized) on test samples.
- Display top-1 predicted label for each input.
- Compare with true label.

Questions:

- How accurate is the model with only random weights?
 - What are the limitations without training?
-

Deliverables:

- Jupyter Notebook with implementations of all layers
- Visualizations: input image, feature maps, pooled maps
- One forward pass showing output predictions

Bonus:

- Implement batch-based processing
- Visualize feature maps for different digits
- Add backpropagation for training loop

Prepared by: Asad Ahmed - AI Engineer @IR Solutions

Date: August 2025